

AWS Well-Architected Framework Review – Web Content Filtering Proxy Solution

Architecture Overview

The target application is a **man-in-the-middle web proxy** designed for advanced web content filtering. It intercepts user web requests and **deconstructs HTML content** into components, analyzes and classifies text (including semantic analysis and sentiment), then **reconstructs the sanitized HTML** before delivering it to the end user. Key aspects of the architecture include:

- **Global Edge Distribution:** User traffic is routed through Amazon **CloudFront**, providing a global entry point with low-latency delivery and TLS termination. CloudFront is fronted by a custom DNS domain with an AWS Certificate Manager TLS certificate, ensuring secure access. This design also makes it feasible to direct users to multiple AWS regions for an **active-active** multi-region setup if needed (leveraging CloudFront or Route 53 routing for low latency and failover).
- **Proxy Tier (EC2 Auto Scaling):** CloudFront directs requests to a regional **Network Load Balancer (NLB)**, which distributes to a fleet of EC2 instances running the custom proxy software. These EC2 proxies are **stateless** (no session stickiness needed) – they simply forward requests to backend services and return filtered content. The EC2 fleet is managed in an **Auto Scaling Group (ASG)** using a custom **AMI** (built via CI pipeline) for the proxy. The ASG spans multiple **Availability Zones**, and the NLB health checks ensure only healthy instances receive traffic. The proxies can scale out horizontally to handle load (and scale in when idle), including using **Spot Instances** for cost savings since proxies are stateless and fault-tolerant (Spot interruptions simply trigger replacement instances) ¹. This scalable, multi-AZ design avoids any single point of failure and can automatically recover from instance or AZ disruptions.
- **Serverless Microservices:** The filtering logic is broken into several serverless components (microservices), each implemented as an **AWS Lambda** function (with FastAPI/Python code). These include the **Man-in-the-Middle API** (orchestrating the overall flow), an **HTML Service** (to parse and rebuild HTML), a **Semantic Text Service** (for text classification, sentiment analysis, possibly leveraging external AI APIs), an **HTML Graph/Storage Service**, and a **Caching Service**. Each Lambda function is deployed behind a CloudFront distribution (and possibly Amazon API Gateway or Lambda URLs) with its own domain (e.g. `cache.dev.mgraph.ai`, `cache.qa.mgraph.ai`, `cache.prod.mgraph.ai`, etc., for different environments). The Lambdas are fully **stateless** and rely on external storage or other APIs for any stateful data.
- **Data Storage (Amazon S3 as a Database):** The solution notably uses **Amazon S3** as the primary data store for stateful data (no traditional database). S3 buckets (with versioning enabled) hold cached content, logs, and any persistent data via a custom **“MemoryFS”** layer. The Caching Service and other Lambdas read/write to S3 (abstracted through the MemoryFS module) as if it were a filesystem or key-value store. S3’s inherent scalability and durability (11 nines) provide a simple, highly durable “database.” This approach eliminates managing a separate database service, though it means data access is eventually consistent and subject to S3 performance

characteristics. The S3 buckets have versioning and delete protections enabled to prevent accidental data loss.

- **CI/CD and Environment Isolation:** All components (proxy AMI, Lambda functions code packaged as PyPI modules, infrastructure-as-code templates) are built and deployed via an automated CI/CD pipeline. Separate deployment stages exist for **Dev, QA, and Prod** environments, each with its own set of CloudFront endpoints (e.g. `*.dev.mgraph.ai`, `*.qa.mgraph.ai`, `*.prod.mgraph.ai`). This allows testing in isolation and promoting to production with confidence. Unit test coverage is high (often 90%+), improving code quality and reliability in each release. Deployment to AWS is scripted (using CloudFormation or similar) so that the entire stack can be brought up or down reproducibly. (In fact, the team plans to implement scheduled **environment teardown** after business hours to save cost, since the system can be redeployed quickly in the morning.)
- **Logging and Monitoring:** AWS **CloudWatch** collects logs from the Lambda functions (and presumably from EC2 via CloudWatch agent or NLB access logs). Application-level detailed logs are also written to S3 for later analysis. However, at present there are **no CloudWatch dashboards or alarms** set up for this system – monitoring is ad-hoc (logs exist, but no active alerting).
- **Security and Identity:** Each Lambda and EC2 runs under a specific **IAM role** with scoped permissions (following least-privilege principles to some extent). There are no user-managed servers; all infrastructure is in AWS, so network access can be controlled with security groups and the NLB (for example, only CloudFront or certain sources can call the proxies). **Secrets management** is a current gap: service credentials or API keys are provided via environment variables which are populated from **GitHub Secrets** during deployment (CI pipeline injects them). This means secrets are not managed in AWS at runtime (not using AWS Secrets Manager or Parameter Store yet). Authentication between services is done with API keys passed via environment config and headers (not a centralized OAuth or Cognito – since this is an internal microservice auth).
- **External Dependencies:** Heavy AI/LLM tasks are offloaded to external services (AWS Bedrock, OpenAI API, Groq, or OpenRouter) rather than running large models on these Lambdas. This keeps the architecture lean – the Lambdas just call external APIs for AI functions, which is aligned with the idea of “**democratizing advanced technologies**” by consuming them as managed services/API instead of self-hosting models ² ³.

In summary, the architecture is **largely serverless and stateless**, with a small always-on footprint (the proxy EC2 tier) that can scale dynamically. It is globally accessible via CloudFront, and designed for high availability and scalability. The following sections will review this architecture against AWS's Well-Architected Framework pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability. For each pillar, we identify strengths, weaknesses, and recommendations.

Operational Excellence (OE)

Operational Excellence focuses on running and monitoring systems to deliver business value and continually improving processes ⁴. In the current setup, the team has implemented solid **CI/CD pipelines**, high test coverage, and clear separation of dev/QA/prod which are positives for operational excellence (facilitating frequent, reliable releases). The **infrastructure is codified** (AMI builds, Lambda

deployments via pipeline), which indicates changes are automated and repeatable. This reduces manual effort and errors during deployments or scaling operations.

However, there are some notable gaps in operational practices:

- **Runbooks & Playbooks:** The team has **no documented runbooks or playbooks** for common operations or incident response. At the moment, knowledge is likely in the heads of the developers or in extended documentation, but formal step-by-step guides don't exist. AWS best practices recommend having **runbooks** for routine procedures and **playbooks** for incident investigation. Runbooks are "codified processes that provide consistent outcomes no matter who uses them," and should include steps for handling errors and escalations ⁵. The absence of runbooks could slow down response times and introduce variability in handling incidents or maintenance tasks. It's advisable to create runbooks for things like restarting the proxy service, redeploying Lambdas, handling a regional failover, updating secrets, etc., and store them in a central repository accessible to the team ⁵. Over time, these can be automated with AWS Systems Manager or scripts to reduce manual toil.
- **Monitoring & Alerting:** While logs are collected (in CloudWatch and S3), there are **no CloudWatch dashboards or automated alarms** configured. This is an operational risk because the team might not be promptly aware of issues like increased error rates, latency spikes, or failed Lambda invocations. According to AWS's operational excellence practices, teams should **communicate operational status through dashboards and notifications** tailored to audiences, so that issues can be acted upon promptly ⁶. It is recommended to set up CloudWatch metrics for each key component (e.g., Lambda errors/throttles, latency; EC2 CPU/memory; NLB 5xx errors; CloudFront Cache hit rate, etc.) and define **alarm thresholds** for critical metrics. For example, an alarm on "Proxy fleet 5xx errors > N per minute" or "Lambda timeout errors > 0" could alert on-call engineers. CloudWatch Alarms can send to SNS or PagerDuty/email so the team is notified immediately. *Monitoring metrics should be used to raise alarms when thresholds are breached* ⁷ – this enables automated response or human intervention quickly. Without alarms, an issue might go unnoticed until users report it, impacting customer experience.
- **Operational Visibility:** Beyond raw metrics, having **visual dashboards** (e.g., CloudWatch Dashboards or third-party APM tools) would help operators quickly assess system health. Dashboards can show real-time and historical trends for traffic, latency, error rates, and resource utilization. This aids in both incident response and in continuous improvement (spotting trends to optimize). AWS Well-Architected suggests providing dashboards for different audiences (ops team, dev team, business, etc.) to communicate system health at a glance ⁸. Setting up a few key CloudWatch dashboard widgets (or using services like AWS X-Ray for tracing, if applicable) would greatly enhance visibility into this distributed system.
- **Incident Response Preparedness:** There is no mention of **game days** or simulated incident drills. The team has not conducted full resilience or failover drills yet. Regular **game days** (simulated events in production or test environments) are a recommended practice to test how both the architecture and the team processes perform under failure scenarios ⁹. For example, a game day might simulate "What if the primary region goes down?" or "What if the cache S3 bucket is accidentally made unavailable?" and then practice the recovery steps. This helps ensure runbooks are effective and team members are familiar with them ¹⁰. Given the current lack of runbooks, game days would also highlight that gap. Instituting game days (perhaps quarterly) would improve confidence that the team can handle incidents smoothly and meet RTO/RPO targets.

- **Continuous Improvement:** The architecture is evolving (e.g., the plan to automate environment teardown to save costs is an improvement in process). The team should continue to **evolve operational processes** by collecting metrics from incidents and deployments to identify areas to fix. For instance, after any incident or even a near-miss, a *post-incident analysis* (blameless retrospective) can generate action items (maybe to add an alert or improve a runbook). This creates a feedback loop for operational excellence.

Recommendations – Operational Excellence:

- **Establish Runbooks/Playbooks:** Create a repository of runbooks for common operational tasks and incident scenarios. Ensure they are centrally stored, kept up to date, and cover error handling and escalation procedures ⁵. Start with the highest priority processes (deployments, outage recovery, rotating secrets, etc.) and expand over time. Encourage the team to follow these documents during operations to ensure consistency.
- **Implement Monitoring Dashboards:** Set up CloudWatch Dashboards or equivalent to track key metrics across the system. Include graphs for request rates, error counts (4xx/5xx), latency (p95, p99), CPU/memory of EC2 instances, Lambda concurrency, etc. This will give quick insight into system health for on-call staff or during debugging ⁸.
- **Set up Alerts/Alarms:** Define CloudWatch Alarms for critical conditions (e.g., high error rate, failed health checks, low availability of instances in ASG, high latency, exhausted Lambda concurrency, etc.). Configure notifications (SNS/email/Slack) so that the team is alerted **immediately** when thresholds are breached. Ideally, some alarms could trigger automated actions as well (for example, if an instance is unhealthy, ASG already replaces it; if a Lambda is erroring, an alarm could disable a route, etc.) – but at minimum, notify humans to respond ⁷.
- **Practice Incident Response:** Conduct **game days** to simulate failures and test the team's response procedures ⁹. For example, simulate an AZ outage (maybe by manually stopping instances or invalidating the cache) and see if the system self-recovers and if the team's runbooks cover the steps. Use these drills to refine runbooks and to train team members in handling the unexpected.
- **Leverage AWS Tools:** Consider using AWS tools like **AWS CloudTrail, Config, or AWS Health Dashboard** to augment operational awareness. CloudTrail can audit changes/deployments (useful if an operational issue is caused by a deployment change). AWS Health events can alert you to AWS service issues affecting your stack. Additionally, AWS Systems Manager Incident Manager could be looked at in the future as you mature, for coordinating incident response.

By improving in these areas, the team will enhance operational excellence, ensuring the system is easier to operate, issues are detected and resolved faster, and processes evolve continuously.

Security

The security pillar focuses on protecting information & systems, managing access, and detecting/responding to threats ¹¹. In this architecture, there are several security considerations:

Positives: The deployment uses **IAM roles** for EC2 and Lambda with limited scopes. That suggests an effort toward the principle of **least privilege** – ensuring components only have the permissions necessary for their function ¹². For example, the Lambdas likely each have roles that allow access only

to the specific S3 bucket paths or services they need (and nothing more). The EC2 proxies might have an IAM role that permits fetching from S3 (for cache or config) and calling the Lambda/API endpoints, but not broad admin access. Using IAM roles (instead of long-lived AWS keys) is good practice and credentials are short-lived by default. Network security is partly achieved by using CloudFront and NLB – the EC2 instances are not exposed directly to the public internet except through the controlled NLB (which could have security group rules or NACLs). Also, all traffic is encrypted via HTTPS (CloudFront to users, and likely CloudFront to origin using TLS as well). S3 buckets have versioning and likely enforce encryption at rest (it's not stated, but by default S3 encrypts new objects and the team can enforce bucket policies to require it). These are all good security measures if in place.

Areas to Improve:

- **Secrets Management:** Currently, secrets (such as API keys for inter-service calls or credentials for external APIs) are stored in **GitHub Secrets** and injected into Lambda environment variables at deploy time. While GitHub encrypts secrets at rest and in transit, this approach has some drawbacks in an AWS environment. The secrets end up in Lambda configuration (plaintext environment vars) which could be accessed by anyone with Lambda console access or if logs print them accidentally. A better approach is to use AWS native secrets management like **AWS Secrets Manager or SSM Parameter Store**. AWS Secrets Manager in particular provides secure storage with encryption, rotation, fine-grained access control, and audit logging of secret access ¹³. It acts as a managed “**credential vending machine**” where Lambdas can fetch secrets at runtime securely (with IAM policies controlling access) ¹⁴ ¹³. This removes the need to store secrets in CI/CD or code. The team should consider migrating sensitive configuration (API tokens, etc.) to AWS Secrets Manager or Parameter Store. Lambdas can retrieve them on startup (possibly caching them in memory for performance). This also simplifies rotation of secrets (Secrets Manager can auto-rotate certain creds) and ensures that access to secrets is logged/audited ¹⁵. At a minimum, if staying with the current approach, ensure that the CI/CD logs do not leak secrets and that environment variables are encrypted (Lambda env vars are encrypted at rest by AWS, but decryption could be seen by someone with the right privileges).
- **Least Privilege & IAM Governance:** While IAM roles are used, the prompt acknowledges they “can be improved and locked down.” It's worth reviewing all IAM policies attached to these roles to tighten them. Follow **grant least privilege access** guidelines – each role should have only the specific actions on specific resources it *needs*, and nothing more ¹². For example, if the HTML Lambda only needs `s3:GetObject` on a certain bucket path, it should not have full `s3:*` on all buckets. Use IAM condition keys (like restricting by source VPC or required encryption) where appropriate. Also, consider **scoping access by environment** – e.g., dev Lambdas shouldn't access prod resources. AWS recommends regular IAM access reviews to avoid privilege creep ¹⁶ ¹⁷. This could be done with tools like IAM Access Analyzer to find unused permissions. Given that multiple accounts or at least multiple stages are in use, ensure that the CI/CD deployment roles themselves are secure (those GitHub actions likely assume an AWS role – that role should be limited to deploying only the needed resources).
- **User Authentication & Authorization:** The system's services authenticate to each other via static API keys set in env variables (as noted). This is a basic shared-secret approach. If this system will be used by external customers or across multiple clients, a more robust authentication/authorization might be needed (for instance, using Amazon Cognito or issuing JWTs for requests, etc.). However, if it's internal within a single trust boundary, the API key approach can be acceptable, though rotating those keys periodically is a good practice. Also, CloudFront can integrate with AWS WAF for an additional layer of security (to filter out malicious

patterns, prevent DDoS, etc., although AWS Shield Standard already protects CloudFront and NLB at a baseline level).

- **Security Logging and Response:** There's no mention of enabling **AWS CloudTrail** or other security logging beyond application logs. It's highly recommended to ensure CloudTrail is enabled in all regions for the account, logging to an S3 bucket (with access locked). This will record all console/API actions, which is crucial for incident investigations. Additionally, consider AWS Config rules or Amazon GuardDuty for threat detection. GuardDuty in particular would alert on any anomalous behavior (like if an EC2 instance was compromised and making strange calls, or if credentials leaked). Setting up some basic **alarms for security events** (like GuardDuty findings, or Config detecting a public S3 bucket etc.) would enhance the security posture.
- **Infrastructure Protection:** The EC2 proxies should be running in private subnets (accessible only via the NLB), and their security groups should restrict egress if possible (maybe only allow them to call AWS services or known external APIs for AI). Applying patches to the AMI regularly is important since these are long-running instances (even though recycled often, ensure the base AMI is updated with security patches frequently via the pipeline). Use AWS Inspector or at least run baseline security scans on the AMI occasionally. The Lambda functions run on AWS-managed VMs, so less concern there, but ensure the code (which is open source) doesn't have known vulnerabilities (maybe integrate **Dependabot** or similar to catch library vulns). Also, S3 being the "database" means securing it is paramount: use S3 bucket policies to only allow access from the specific IAM roles, enable default encryption, maybe enable Access Logs or CloudTrail data events on S3 to see if any unusual access.
- **Secrets in Code:** A related concern – ensure no secrets (like AWS keys or API tokens) are hardcoded in the Lambdas or AMI scripts. The team likely avoids this by using env vars, but it's worth scanning the code (with tools or manual review) for any accidental secret commits. AWS CodeGuru (or open-source truffleHog, etc.) can detect secrets in code repositories ¹⁸.

Recommendations – Security:

- **Adopt AWS Secrets Manager for sensitive config:** Migrate application secrets (API keys, credentials) to **AWS Secrets Manager** and retrieve them at runtime. This provides built-in encryption, rotation, and auditing of secret access ¹³. It will remove reliance on GitHub for secret storage and ensure that even within AWS, only authorized components can access the credentials (via IAM policy).
- **Tighten IAM Policies:** Review all IAM roles and policies for the Lambda functions and EC2 instances. Remove any wildcard permissions and scope them to the minimal required actions/resources ¹². Implement a regular audit (perhaps quarterly) of IAM permissions, using AWS IAM Access Analyzer and Access Advisor to find and revoke unused access ¹⁶ ¹⁹. This will uphold the principle of least privilege and reduce risk if any credentials are compromised.
- **Enable Advanced Logging & Monitoring for Security:** Ensure AWS CloudTrail is enabled and monitoring all API calls. Consider turning on **CloudTrail Insights** for detecting anomalous activity. Enable Amazon GuardDuty for threat detection (it's low effort to enable and provides valuable alerts on things like port scanning, unusual data exfiltration, etc.). Set up notifications for any critical GuardDuty findings. Also, configure AWS Config with security-focused rules (for example, to ensure S3 buckets are not public, IAM keys are rotated, SGs aren't wide open, etc.).

- **Web Application Security:** Given the application deals with user web content, make sure the proxy sanitization itself is robust to malicious content (this is an app concern, but part of security posture – e.g., avoid XSS by proper filtering). Use AWS WAF on the CloudFront distribution to block obvious bad actors or patterns if applicable (WAF could mitigate bots or large floods of unwanted traffic, augmenting CloudFront’s built-in protections).
- **Secret Rotation Plan:** Implement a schedule to rotate API keys and secrets regularly. If using Secrets Manager, you can automate this. If not, at least use the CI/CD to roll keys (and update environment variables) periodically so that if a secret did leak, it has a limited lifetime.
- **Incident Response Plans for Security:** Develop a simple incident response playbook specifically for security events (e.g., if a key is compromised, or if GuardDuty flags an EC2 instance). This should outline steps to contain (maybe tearing down and redeploying resources, revoking credentials, etc.). **Routinely practice security incident response** as well, possibly as part of game days ²⁰ ²¹ . This ensures the team is ready to handle, say, a suspected breach with speed and rigor.

Overall, by improving secrets management and tightening access control, plus leveraging AWS’s security services for monitoring, the solution will be much more robust against threats while still enabling the needed functionality. The goal is to **protect data and systems but also detect and respond to incidents quickly** ²² ²³ .

Reliability

Reliability is about a system’s ability to recover from failures, scale to meet demand, and maintain availability ²⁴ . The described architecture already incorporates several reliability best practices:

Strengths:

- **Multi-AZ High Availability:** The critical components (the EC2 proxy tier, and presumably the Lambdas which are multi-AZ by nature of AWS Lambda) are spread across multiple Availability Zones. The EC2 Auto Scaling Group is configured across at least two AZs, and the NLB will route to any healthy instance in any AZ. If one AZ has an outage or an instance fails, the load balancer and ASG work together to route traffic to the others and launch new instances as needed. This follows the fundamental AWS guidance to **distribute workloads across multiple AZs** to avoid single points of failure ²⁵ ²⁶ . The benefit is improved resiliency against data center failures – as AWS notes, using multiple AZs can protect against power or network issues impacting one AZ ²⁷ ²⁶ . The design is already stateless such that any proxy instance can handle any request, which is ideal for horizontal scaling and redundancy.
- **Auto Recovery and Scaling:** The use of an **Auto Scaling Group** with health checks means that if an EC2 instance becomes unhealthy (failing the custom health check endpoint), the ASG will terminate it and launch a replacement automatically. This provides automatic recovery at the infrastructure level without operator intervention. The team has also parameterized scaling policies (scale out on traffic spikes, and scale in when low). This “**automatic failure recovery**” is a key design principle for reliability ²⁸ . Additionally, being able to scale horizontally (adding more instances) increases the overall availability and capacity of the system ²⁹ , allowing it to handle bursts in traffic by distributing load.

- **Stateless Services & Data Durability:** The stateless nature of the proxies and Lambdas simplifies recovery – you can always recreate instances or reroute without worrying about session loss. **State is stored in S3**, which is a highly durable service (99.999999999% durability). S3 by default stores data across multiple AZs in a Region, meaning even if one AZ goes down, the data remains available from another AZ ³⁰. The team has enabled **versioning** on the S3 buckets and disallowed permanent deletes, which protects against accidental deletions or overwrites. This is a form of backup/version control for data. In effect, S3 (especially with versioning) serves as an always-on backup of itself. There's no traditional database that could be a single point of failure – using S3 and Lambdas (which are inherently multi-AZ) means core data and compute are highly available by design.
- **Multi-Region Possibilities:** The architecture mentions the ability to deploy in **3-4 different AWS Regions** and use a global DNS or CloudFront to distribute traffic among them. This would be an active-active multi-region deployment. If implemented, that dramatically increases reliability because even a regional outage could be mitigated by another region taking over traffic. CloudFront itself is a global service and can have origins in multiple regions (though CloudFront doesn't natively failover between origins in different regions without some custom configuration or Route 53 help ³¹). It's not clear if multi-region is currently live or just potential, but the fact that AMIs can be built per region and the infrastructure can be redeployed easily means the team could achieve **disaster recovery** in another region if needed (possibly with manual intervention at this point).

Areas for Improvement:

- **Testing Recovery and DR:** The team noted they have *not performed full resilience, failover drills*. While the architecture is built for high availability, it's crucial to **test those assumptions**. For example, simulate an AZ failure to see if the ASG truly balances and the system stays up. Or simulate a region failure (if multi-region is configured, ensure Route 53 latency or failover routing is set up; if multi-region is not yet live, consider at least a plan for it). AWS Well-Architected suggests regularly **testing recovery procedures** – e.g., intentionally shutting down components to verify that alarms trigger and auto-healing works ³². It's better to find any misconfigurations (like if the health check path is wrong, or ASG scaling limits are too low, etc.) in a controlled game day than during a real incident. Similarly, test restoring data from S3 versioning in case of accidental corruption (ensure you know how to recover an old version quickly if needed).
- **Fault Isolation & Redundancy:** Ensure that **all components** have redundancy. For instance, Lambdas are redundant by nature (multiple executions handled by AWS), S3 is redundant, EC2 proxies are redundant. The DNS and CloudFront are also redundant (CloudFront uses many edge locations). One component not discussed is the CI/CD pipeline – if deployment is critical, consider using multiple AWS accounts or backing up CloudFormation templates, etc., so that you can deploy to a new account/region if one is down. Also, consider **S3 cross-region replication** if a truly multi-region active-active or active-passive setup is desired for the data. Although S3 is multi-AZ, an AWS region outage could make the data temporarily inaccessible; having a cross-region replicated bucket (with proper failover logic) could protect against that, at the cost of complexity and consistency considerations.
- **Health Checks & Graceful Degradation:** The proxy has a health check (for the NLB/ASG to use). It would be good to monitor the health check success rate. If an instance starts failing health checks frequently (perhaps due to heavy load or memory issues), that could indicate scaling needs or a bug. Additionally, think about graceful degradation strategies: if, say, the semantic

classification service is down, does the proxy still serve the page without that analysis (perhaps with a default or an error banner), or does it fail the whole request? Building in timeouts and fallbacks (maybe skip a step if a non-critical service is unavailable) would improve user experience continuity. This is an application-level reliability consideration.

- **Monitoring Reliability Metrics:** Overlaps with Ops, but specifically, monitor SLA-related metrics. For example, what is the current uptime of the service? Are there any error patterns during deployments (maybe brief outages when deploying new Lambdas or AMIs)? Gathering this data allows you to decide if you meet your reliability goals or if you need improvement (like adding rolling deployments to avoid any downtime).
- **Use of AWS Reliability Services:** The architecture doesn't mention AWS Backup, AWS Restore or similar – for S3, versioning covers a lot, but for other state (like if there were any EBS volumes or such), ensure backups. In this case, EBS is mainly for EC2 instances which are ephemeral, so backups not needed aside from the AMI (which is already an image backup). If any configuration is stored only in certain places, back it up (for example, if CloudFront or Route 53 config is critical, use infrastructure as code so it's always reproducible – which they do via CloudFormation templates, it seems).

Recommendations – Reliability:

- **Conduct Regular Recovery Drills:** Test the system's resilience by simulating failures. For instance, terminate an EC2 instance manually to verify the ASG replaces it and that the failover is seamless to users. Schedule **game days for failure scenarios** (AZ down, Lambda failure, dependency unavailable) and practice the team's response ³³. This will validate that your alarms, recovery procedures, and architecture design actually achieve the desired RTO (recovery time objective).
- **Implement Multi-Region Disaster Recovery (if required):** If the business demands very high availability (e.g., 99.99% uptime), plan for a **multi-region deployment**. This could be active-active (serve users from nearest region via Route 53 latency routing or CloudFront with multiple origin sets) or active-passive (with a standby region to fail over to). Keep in mind data consistency – you may need to replicate S3 data to the secondary region (S3 Cross-Region Replication can help). Multi-region adds cost and complexity, so weigh it against the reliability requirements ³⁴ ³⁵. If multi-region is not pursued now, at least have a pilot-light plan: e.g., you could spin up in a new region within X hours if Region A fails, using your automated deployment scripts.
- **Improve Fault Monitoring:** Ensure that the health check endpoints are robust and that failing a health check leads to instance replacement. Perhaps push custom metrics like "successful page filter rate" to CloudWatch – a drop could indicate a problem even if instances are technically "healthy." Use AWS Personal Health Dashboard and maybe set up alerts for any AWS service disruptions that could affect your components (for example, if Lambda in a region has an outage, you'd want to know).
- **Versioned and Immutable Infrastructure:** You're already making immutable AMIs for EC2 and deploying Lambdas via versioned packages – continue this practice as it aids reliability (you can roll back to a known-good version if a deployment causes issues). Use CloudFront's support for **staging distributions** or canary tests if you ever need to test changes gradually (for example, Lambda@Edge or CloudFront behaviors can be tested on a percentage of traffic).

- **Chaos Engineering:** As a next-level practice, you might introduce automated chaos testing (using tools like AWS Fault Injector Simulator) to randomly terminate instances or introduce latency in a controlled way. This can further validate the system's resilience.

By proactively testing and planning for failures, and possibly extending redundancy to multi-region, the solution will achieve a robust reliability stance. It's already well on its way with stateless, auto-healing design – the next steps are mostly about validating assumptions and adding layers for extreme scenarios.

Performance Efficiency

Performance Efficiency is about using IT resources efficiently and scaling to meet demands effectively, as load or tech evolves ³⁶. The architecture exhibits several strong performance efficiency characteristics:

- **Serverless and Scalable Components:** The use of AWS Lambda for the majority of components means the compute can **automatically scale** to handle throughput. Lambda will invoke more instances in parallel as requests increase, without the team needing to manage those servers. This fits the design principle of **"use serverless architectures"**, which eliminates the operational overhead of managing servers and can dynamically match capacity to demand ³. It ensures that if, for example, the semantic service suddenly needs to handle 1000 requests, AWS will simply allocate more Lambda instances (up to concurrency limits) to maintain performance. There's effectively no fixed ceiling (other than account limits, which can be raised) – thus high scalability.
- **Global Edge Network:** By fronting services with Amazon CloudFront, the architecture benefits from CloudFront's edge caching and network optimizations. CloudFront can cache static content and even dynamic content for a short time if appropriate, reducing load on origin and improving user latency (though in this use-case, pages are personalized filtered content, so caching might be limited to repeated requests for the same page by multiple users if any). Still, CloudFront provides fast TLS handshakes and uses AWS's backbone to route requests, which likely lowers latency compared to hitting an EC2 directly. Additionally, if multi-region is employed, CloudFront (or Route 53 latency routing) ensures users connect to the nearest region, achieving the principle **"Go global in minutes"** to reduce latency ³⁷.
- **Efficient Instance Usage:** The team has done **performance benchmarking** and identified the most optimized EC2 instance type for the proxy (balancing CPU, memory vs cost). This is crucial – picking the right instance family (perhaps a compute-optimized C6g or similar, or memory-optimized if needed) ensures better throughput per cost. They also leverage **Spot Instances** for the EC2 tier, which provides the same performance as on-demand but at much lower cost. Spot is suitable here because the proxies are stateless and can handle interruption (ASG will replace an interrupted instance). Using Spot might occasionally introduce variability (if instances are reclaimed, new ones spin up, which could have a slight warm-up time), but overall the system remains performant while optimizing cost. Spot Instances are **ideal for fault-tolerant, stateless workloads like web servers and API endpoints** ¹ – exactly the case here. The design already accounts for that with graceful handling of instance churn.
- **Offloading Heavy Work:** All large ML computations are offloaded to external services (AWS Bedrock, etc.). This means the solution avoids doing heavy CPU/GPU work itself, which could slow down response times or require very large instances. Instead, those specialized services

likely handle the load more efficiently (and possibly closer to their data centers, etc.). This approach aligns with “**democratize advanced technologies**” – using managed ML services rather than building your own GPU infrastructure ² . It also leverages **mechanical sympathy**, choosing the right platform for the job (e.g., using OpenAI API for NLP tasks vs trying to run an ML model on Lambda which would be impractical) ³⁸ .

- **Asynchronous Processing Option:** The description hints that some HTML filtering can be done either in real-time or via offline batch. If the design can process content offline (perhaps pre-crawling and caching certain pages), that could improve perceived performance for users (serving cached results instantly). The caching service using S3 might already be storing processed output, so repeated requests for the same content can be served faster from cache instead of recomputation. That's a form of **performance optimization via caching** – reducing redundant work and speeding up responses. We don't have metrics, but presumably if a page has been processed and cached, the proxy can retrieve the sanitized version from S3 much quicker than recalculating everything.

Potential Concerns / Improvements:

- **S3 as Database Performance:** While S3 is highly scalable, using it as a primary datastore for small, frequent reads/writes (like a cache or session store) can have performance implications. S3 has achieved strong read-after-write consistency now and can handle high throughput, but there is latency overhead (tens of milliseconds for each GET or PUT). If the application frequently reads many small objects during a single request, that could add latency. It wasn't detailed how the caching is used in request flow – if each user request triggers multiple S3 reads (for pieces of HTML or logs, etc.), that could slow things down. The team should monitor the latency impact of S3 operations. In some cases, introducing an **in-memory cache layer** (like an Amazon ElastiCache/Redis or using Lambda's ephemeral memory or API Gateway caching) for very hot data could speed up performance. However, this adds complexity and cost, so it's a trade-off. If current performance benchmarks indicate response times are within acceptable range, S3 may be sufficient. Just be mindful that S3, being eventually consistent (though now strongly consistent on writes to new keys), might not reflect data across regions instantly if multi-region is used, and its **throughput scales with prefixes** – so design your S3 key naming to avoid bottlenecks (the modern S3 can auto-scale, but prefix distribution is still a consideration for optimal performance).
- **Cold Start and Concurrency:** Lambda functions can have cold-start latency, especially if they are large packages or not invoked frequently. If certain microservices (like the semantic analysis) are not used often and then suddenly get traffic, there could be a noticeable cold start delay. Optimizing Lambdas for performance would include things like minimizing package size, perhaps provisioning concurrency for critical ones if needed (to keep an instance warm), and using appropriate memory sizing (since more memory = more CPU for Lambdas, which can speed execution). Keep an eye on any such issues from CloudWatch metrics (e.g., see if p95 latency for a service is much higher than p50, which could indicate cold starts).
- **Concurrency Limits and Throttling:** The Lambdas by default have a concurrency limit per account. If the traffic grew massively, ensure those limits are raised accordingly to avoid throttling. The same goes for API Gateway (if used) or CloudFront – though CloudFront can handle huge scale by design. The EC2 ASG should also have its max capacity high enough to not throttle scaling. It sounds like they configured it to scale up to 20–30 instances which is good; just ensure there's a plan if you ever needed even more (e.g., perhaps adding another region's capacity or increasing instance size).

- **Performance Testing and Optimization:** The team has done quite a lot of performance benchmarking already, which is excellent. They should continue to profile where the time is spent in processing a page. If the bottleneck is external (like waiting on an OpenAI API response), not much can be done except perhaps parallelizing calls or using faster models. If any bottleneck is internal (like assembling the HTML or doing the semantic analysis in Python code), consider optimizations such as using more efficient algorithms, or even offloading some parts to C++ libraries or AWS Lambda's ability to run in parallel (e.g., use AWS Step Functions to parallelize calls if needed). Given the open source nature, the community might also contribute performance improvements.
- **Content Delivery Optimization:** CloudFront can compress content (gzip/Brotli) and use HTTP/2 to improve client performance. Ensure those are enabled so that users get the content quickly. Also, if the content is large (lots of HTML), consider if it can be streamed or delivered gradually. But since this is a web proxy, likely it streams as it builds.

Recommendations – Performance Efficiency:

- **Continue Leveraging Serverless & Managed Services:** The architecture already aligns well with using managed services (Lambda, CloudFront, S3) for performance and scalability. Continue to review new AWS offerings – for example, if AWS releases a new service or instance type that could improve performance (such as AWS Graviton-based instances which often have better price/performance), be ready to test those. Regularly **review and evolve your technology choices**, as per AWS guidance to take advantage of the continually evolving cloud offerings ³⁹.
- **Optimize S3 Usage:** Monitor the latency impact of using S3 as your database. Use S3 **Transfer Acceleration** or multipart downloads for large objects if needed, but for many small objects consider grouping some data or using caching. If you find S3 access is a bottleneck, evaluate adding an in-memory cache (e.g., a Redis cluster for super-fast retrieval of hot items) or switching certain high-frequency data to a DynamoDB table (which has single-digit ms latency). DynamoDB could complement S3 for very latency-sensitive key-value lookups (with caching service writing to both perhaps). This is only if needed – if current latency is fine, keep it simple with S3.
- **Profiling and Reducing Cold Starts:** Ensure that critical Lambda functions are as lightweight as possible. For example, avoid large dependencies in the deployment package and initialize outside the handler where possible. You could also consider **Provisioned Concurrency** for Lambdas that must always be responsive (though that incurs cost). Given this is a user-facing service, a small concurrency buffer on the API Lambdas might help maintain consistently low latency.
- **Parallelize and Batch Work:** If not already, consider fetching and processing content in parallel when possible. For instance, if the HTML deconstruction can send parts of text to the semantic service in parallel rather than sequentially, that could reduce overall processing time (provided the Lambda concurrency allows it). AWS Step Functions or just asynchronous Lambda calls could coordinate that. Also, if multiple external AI API calls are needed per page, try to do them in parallel threads or async (depending on the code environment).
- **Continue Performance Testing:** Incorporate performance tests into your pipeline or regular schedule. Every time a new version is deployed, run a suite of load tests to ensure latency and

throughput are within expectations. This catches any regression early. Over time, usage patterns may change (e.g., longer pages, more concurrent users); be proactive in testing for future scale.

In summary, the system is well-optimized through serverless design, global distribution, and smart use of external services. The recommendations above are mostly about fine-tuning and staying vigilant as the system scales. With these in place, the architecture should continue to perform efficiently under increasing load and as requirements evolve.

Cost Optimization

Cost Optimization in AWS involves delivering outcomes at the lowest price point by eliminating waste and selecting the most cost-effective options ⁴⁰. This solution already employs several cost-efficient strategies:

- **Heavy Use of Serverless and On-Demand Services:** By using AWS Lambda and CloudFront, the architecture benefits from a **pay-per-use** model. There are no idle server costs for those components – if usage is low, costs scale down automatically (Lambda charges only when run, CloudFront costs scale with traffic, S3 costs with storage/requests). This inherently optimizes cost for variable workloads because you're not paying for over-provisioned capacity. The design to even shut down nearly everything after hours (except storage) takes this further: essentially scaling to **zero** usage cost when no traffic. That's an ideal in cost optimization – **only pay for what you use**. It's worth noting that CloudFront and S3 minimal costs and a minimal EC2 might remain, but those are relatively small.
- **Spot Instances for EC2 Fleet:** The proxies run on Spot Instances, which can be up to 90% cheaper than on-demand prices ⁴¹ ⁴². This is a major cost saver for the always-on portion of the stack. The team has configured the system to handle Spot interruptions gracefully (instances are frequently recycled anyway, and stateless design makes interruption a non-issue beyond perhaps a slight reconnection). Using Spot is a well-architected cost practice for fault-tolerant workloads, and the team took care to find the optimal instance size as well (rightsizing). For example, if they determined that **c5.large** instances give the best throughput per dollar and more smaller instances outperform fewer big ones, that's great analysis. AWS recommends such **instance right-sizing** and exploiting Spot for short-lived or flexible workloads ⁴³ ⁴².
- **Auto Scaling (Scaling Down):** The system can scale down to a single small EC2 instance in low traffic, and Lambdas naturally scale to zero if no requests. This elasticity means minimal waste – you're not running 10 servers 24/7 “just in case”; you might be running 1 at midnight and 10 at noon, for example. Additionally, the plan to **delete infrastructure after 6pm** and recreate in the morning for development/QA or even production (if production usage is only daytime, perhaps) is an extreme cost-saving measure. Destroying everything (except keeping the data in S3) means zero EC2/NLB/CloudFront hourly charges during off-hours. As long as automation reliably brings it back, this can cut costs dramatically. (Few production systems do that because of 24/7 needs, but for internal or non-critical environments it's smart.)
- **Efficient Use of AWS Services:** By using S3 as the database, they avoid the cost of running a separate database service (e.g., RDS or DynamoDB costs could be higher for large data or constant throughput). S3 costs are generally low (pennies per GB-month and per 1000 requests). Also, CloudFront has caching which, if effective, could reduce origin calls (less Lambda execution = cost save, less NLB data transfer = save). If Lambdas have repetitive external calls, maybe caching results can save API costs too. The CI/CD automation likely uses AWS CodePipeline or

GitHub Actions – not sure, but any efficient deployment pipeline also reduces manual effort (cost in terms of time).

Areas to look for further cost optimization:

- **Monitoring and Optimizing CloudFront/Data Transfer Costs:** Sometimes a significant portion of cost in such solutions can be data transfer, especially if users download a lot of content via the proxy. CloudFront helps by cheaper rates and caching. Ensure you've enabled the **CloudFront Cache** appropriately (maybe caching common static resources on pages like images, CSS, which don't need filtering) so that those don't hit your origin repeatedly (saving Lambda invocations and EC2 bandwidth). Also, use the CloudFront **cost calculator** or reports to see if any region is driving high costs; maybe adjust TTLs or consider AWS **Global Accelerator** (though that has a fixed cost) if needed for better cost/performance trade.
- **AWS Savings Plans/Reserved Instances:** Spot covers the EC2 mostly. If there are any components that are on-demand and long-running (for example, the NLB hours or if you decide to keep one on-demand instance always as a buffer), consider a Compute **Savings Plan** or Reserved Instances for that baseline. NLB and CloudFront don't have reservations, but Lambda usage if predictable could be reduced via Compute Savings Plan as well (it applies to Lambda). This is a minor gain compared to Spot, but worth evaluating if you have steady-state usage.
- **Optimize Lambda cost:** Lambdas are cost-efficient, but ensure the memory setting is not over-allocated (over-paying per execution) or under-allocated (causing longer runtime which also costs more). There's a sweet spot for each function where cost (memory * time) is minimized. Given testing done, maybe they've tuned this, but it's something to revisit occasionally especially after code changes.
- **Storage Costs:** S3 is cheap per GB, but if the application stores a lot of cached data or logs, ensure you have a lifecycle policy. Data that is old and not likely needed could move to cheaper storage classes (S3 Standard-IA or Glacier for very old logs). The *caching strategy* should consider evicting or aging out old content from S3 if it's not needed, to save space and cost. Versioning on S3 also means old versions pile up; occasionally review if those should be cleaned (maybe after X days if not needed for backup purposes).
- **Centralized Cost Visibility:** Use AWS Cost Explorer and maybe set up **cost allocation tags** for different components (tag resources by environment or microservice) to see where the money is going. The team should set up a **budget alarm** if not already, to get notified if monthly cost exceeds expected thresholds ⁴⁴. This aligns with **proactive cost monitoring** so there are no surprises ⁴⁴.
- **Third-Party Services Costs:** Keep an eye on those external AI API costs as well – they might actually dwarf AWS costs depending on usage! Not part of AWS Well-Architected per se, but from an overall solution perspective, ensure calls to OpenAI etc. are efficient (maybe reuse embeddings or caching results so you don't call the same analysis repeatedly for the same text).

Recommendations – Cost Optimization:

- **Implement Cost Monitoring and Transparency:** Even though costs are low due to Spot/serverless, use AWS Cost Explorer and CloudWatch to **monitor cost metrics**. Set up dashboards or reports that show daily spending on EC2, Lambda, Data Transfer, etc. This will highlight if any

cost category starts growing unexpectedly (e.g., a bug causing excessive Lambda invocations or an increase in data transfer). **Proactive monitoring of cost** ensures you catch inefficiencies early ⁴⁴. For example, a sudden spike in S3 PUT requests might indicate a misbehaving loop – catching it quickly can save money and fix the bug.

- **Leverage AWS Savings where applicable:** Evaluate using a **Savings Plan** for a portion of the workload. If you know a baseline of Lambda/EC2 usage will persist, a 1-year Compute Savings Plan could save ~30% on that portion without affecting flexibility (it will even apply to Fargate or Lambda). Since Spot covers most EC2, focus might be on Lambda and NLB hours (though NLB doesn't have SP, but its cost is usually not huge). This is a nice-to-have if the scale grows.
- **Optimize S3 Costs with Lifecycle Policies:** Configure S3 lifecycle rules for logs and cached objects. For example, logs older than 30 days move to Standard-IA, and to Glacier after 90 days, if you rarely access old logs. If cached HTML content is only useful for, say, a week (since web pages change), consider expiring or overwriting it to avoid accumulating stale data. This will keep storage costs minimal.
- **Review and Right-Size Periodically:** Continue the practice of **right-sizing** instances and services. AWS releases new instance families and sizes often – maybe an **ARM64/Graviton** instance could run your proxy cheaper or faster. Since you can build AMIs, you might try a Graviton (e.g., C7g or similar) if not already. They tend to be cost-effective (up to 40% better price-performance in many cases). Also, if your lambdas are using less memory than allocated, adjust the memory down to save cost (or vice versa if runtime is high, sometimes giving more memory can reduce duration and cost).
- **Use AWS Free/Discounted Tools:** Take advantage of things like the AWS Free Tier where possible (maybe dev environment stays within free tier limits for Lambda/S3 if usage is low). Also, since all code is open source, you might be eligible for credits or cost optimization programs – not an AWS Well-Architected thing, but worth the thought.

The bottom line is the team has been very cost-conscious (Spot instances, scale-to-zero, etc.). By maintaining that mindset and using AWS's cost management tools to continuously inspect and optimize, they will ensure the solution runs at minimal necessary cost while delivering the required performance. In fact, many of the sustainability improvements (next section) also coincide with cost savings, since **maximizing utilization and eliminating idle resources** is good for both environment and budget ⁴⁵ ⁴⁶.

Sustainability (Optional)

(AWS introduced Sustainability as the sixth pillar of the Well-Architected Framework. It focuses on minimizing the environmental impact of cloud workloads ⁴⁷. We include a brief review here given its relevance to cost and architecture decisions.)

This architecture has several sustainability-friendly attributes. By using **cloud services and multi-tenant platforms**, it automatically benefits from the economies of scale and efficiency of AWS data centers. AWS has reported that moving workloads to AWS can **reduce energy usage by nearly 80%** compared to typical on-premises deployments, thanks to high server utilization and more efficient infrastructure ⁴⁸. So, using AWS itself, rather than running dedicated on-prem servers for this proxy, is a big sustainability win.

Specific sustainability considerations in the architecture:

- **Maximize Resource Utilization:** The design **right-sizes instances and uses auto-scaling** to avoid idle capacity. For example, at low load only a tiny EC2 runs, and Lambdas essentially use 0 resources when idle. This means energy is not wasted powering servers that aren't doing useful work. AWS's sustainability guidelines encourage exactly this: **maximize utilization** and eliminate idle resources ⁴⁹. By finding the optimal instance type and using Spot (which utilizes AWS's spare capacity that would otherwise be unused), the solution squeezes the most value and work out of each watt of power consumed.
- **Use of Managed Services:** The heavy reliance on managed services (Lambda, S3, CloudFront) means a lot of resource sharing under the hood. **Managed services** run at massive scale and serve many customers on the same fleet, which leads to higher overall utilization and efficiency. AWS notes that using shared services reduces the amount of infrastructure needed per workload and helps automate efficiency practices (like auto scale down) ⁵⁰. For example, instead of running our own servers 24/7 for an API, by using Lambda, we let AWS fill those servers with work from many customers. This increases hardware utilization and avoids one app's server sitting idle.
- **On-Demand Teardown (Energy Consumption Matching):** Turning off the entire environment during off-peak hours (night) is beneficial from an energy perspective. When the stack is deleted at 6pm, it's not consuming any power overnight. This is effectively matching energy usage to actual demand. It's similar to turning off lights when you leave a room – a simple but effective practice. Many always-on systems don't do this, so this is commendable (assuming the workload truly doesn't need to run 24/7).
- **Efficient Code and Workload:** Offloading large computations to specialized services may also mean those computations are done on optimized hardware (like GPU clusters) that are more energy-efficient for that task than a general CPU would be. Also, by processing content only as needed and caching results, redundant computations are avoided. These all reduce unnecessary CPU cycles, indirectly saving energy.

Possible improvements or considerations for sustainability:

- **Graviton Instances:** AWS Graviton (ARM-based) instances are not just cost-efficient but also energy-efficient, delivering more performance per watt. If the proxy software supports ARM, using Graviton-based EC2s could further reduce the carbon footprint (AWS data shows Graviton2 is up to 2.5x more energy efficient than comparable x86, for instance). This aligns with *anticipating and adopting more efficient hardware* as it becomes available ⁵¹.
- **Optimize Code for Efficiency:** Ensuring the code performing filtering is efficient (both in algorithm and resource usage) can reduce the time and CPU needed for each request, thus using less energy. For example, if there are ways to reduce the complexity of HTML parsing or minimize external calls, that would cut down processing. This is a continuous process – profile the most resource-consuming parts of the app and see if they can be optimized (which often also improves cost and performance).
- **Sustainable Development Practices:** Consider measuring the carbon footprint of the workload using AWS's customer carbon footprint tool. It provides estimates of emissions (based on energy used) for your AWS usage. This could validate the efficiency gains from changes like using Spot

or Graviton. Setting a sustainability goal (like reducing per-request energy by X% next year) can drive innovation that usually also saves money.

- **User Impact:** Since this is a proxy that customers use, making sure it's fast and lightweight also means end-user devices spend less energy waiting or reloading pages. This is a minor angle, but if your service speeds up web browsing by filtering content quickly, users' phones or laptops might use slightly less power (less CPU/rendering time wasted on junk content, for example). AWS sustainability pillar also talks about **reducing downstream impact**, such as not forcing users to upgrade devices often ⁵² – not directly relevant here, but interesting to note.

Recommendations – Sustainability:

- **Right-size and Auto-stop:** Continue the practice of right-sizing instances and automating shutdown during idle times. This maximizes utilization of hardware and **minimizes idle resource waste** ⁴⁹.
- **Adopt Efficient Hardware:** Evaluate moving to AWS Graviton-powered instances for the EC2 proxy tier to gain better performance-per-watt. If performance tests confirm similar or better results, this is a win-win for cost and sustainability.
- **Measure and Improve:** Use the AWS carbon footprint tool to get a baseline of the workload's carbon emissions. Use this data when making architecture decisions (for instance, if running in Region X vs Region Y has differences in energy mix, you might choose the greener region if feasible for multi-region deployments).
- **Managed Services & Automation:** Leverage AWS to handle as much as possible (as you are doing with serverless). AWS data centers are moving toward 100% renewable energy by 2025, so the more your workload is in AWS's court, the more it benefits from that shift ⁴⁸. Also, features like automatic scaling and pay-per-use inherently optimize resource usage.

By consciously designing for efficiency (which the team has largely done) and keeping sustainability in mind for future enhancements, the application not only saves cost but also reduces its environmental footprint. Sustainability efforts often correlate with cost optimization – you **pay for what you use** and avoid waste, which is both economical and eco-friendly ⁴⁵ ⁴⁶.

Pillar Ratings Summary

Finally, here is a summary evaluation of the architecture against each AWS Well-Architected Framework pillar, with a qualitative “rating” and rationale:

- **Operational Excellence: 6/10 – Strengths:** Good deployment automation, environment separation, and high test coverage. **Needs Improvement:** Lacks runbooks and formal incident response plans; no dashboards or proactive alarms in place for monitoring. Improving documentation and automated monitoring will elevate operational maturity.
- **Security: 7/10 – Strengths:** All AWS resources under IAM roles (some least privilege), data in S3 with versioning, TLS everywhere. **Needs Improvement:** Secrets management should be moved to AWS Secrets Manager (current method is not ideal); IAM policies could be further tightened. Also should enhance detective controls (CloudTrail, GuardDuty) and practice incident response.

- **Reliability: 8/10 – Strengths:** Highly available architecture (multi-AZ, auto-scaling stateless servers, durable storage). The system automatically recovers from instance failures and can scale to demand. **Needs Improvement:** Needs regular failure testing (game days) and possibly multi-region deployment for extreme resiliency. Lacks documented recovery procedures and hasn't been battle-tested in disaster scenarios yet.
- **Performance Efficiency: 9/10 – Strengths:** Architecture is globally distributed and serverless where possible, ensuring scalable performance. Efficient use of external specialized services for heavy workloads and caching for repeated tasks. Benchmarking and optimizations have been done (Spot instances tuning, etc.). **Needs Improvement:** Only minor tweaks, like monitoring S3 access patterns or optimizing Lambda cold starts. Overall, the design meets performance needs well and can adapt as load grows.
- **Cost Optimization: 9/10 – Strengths:** Aggressive cost-saving measures in place (Spot instances, scale-to-zero approach, right-sizing). Paying only for necessary resources, with potential to tear down idle environments completely. **Needs Improvement:** Just continue to monitor costs and use tools like lifecycle policies for any long-term data. Possibly leverage Savings Plans for any steady usage. Already extremely cost-efficient for the functionality provided.
- **Sustainability: 8/10 – Strengths:** Efficient use of resources (minimal idle time, high utilization, shared services) aligns with lower energy consumption. Cloud-based deployment benefits from AWS's efficient infrastructure. **Needs Improvement:** Could adopt even more energy-efficient instance types (Graviton) and quantify the improvements. Largely, current cost optimizations also translate to sustainable practices.

Overall, the architecture is **well-designed and aligns closely with AWS best practices** in many areas. The biggest gaps to address are in **operational preparedness and security management**, which can be improved with relatively small changes (process and configuration updates). By implementing the recommendations above, the team can raise the system's Well-Architected posture, achieving a robust, secure, and efficient solution that serves users reliably and can scale for future needs.

1 41 42 43 Cost Optimization Pillar - AWS Well-Architected Framework

<https://docs.aws.amazon.com/pdfs/wellarchitected/latest/cost-optimization-pillar/wellarchitected-cost-optimization-pillar.pdf>

2 3 37 38 39 Performance Efficiency - AWS Well-Architected Framework

<https://wa.aws.amazon.com/wellarchitected/2020-07-02T19-33-23/wat.pillar.performance.en.html>

4 6 7 8 9 10 11 20 21 22 23 24 32 33 36 40 44 AWS Well-Architected Framework

https://pages.awscloud.com/rs/112-TZM-766/images/AWS_Well-Architected_Framework.pdf

5 OPS07-BP03 Use runbooks to perform procedures - AWS Well-Architected Framework

https://docs.aws.amazon.com/wellarchitected/latest/framework/ops_ready_to_support_use_runbooks.html

12 16 17 19 SEC03-BP02 Grant least privilege access - Security Pillar

https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/sec_permissions_least_privileges.html

13 14 15 18 SEC02-BP03 Store and use secrets securely - AWS Well-Architected Framework

https://docs.aws.amazon.com/wellarchitected/latest/framework/sec_identities_secrets.html

25 26 27 30 35 REL10-BP01 Deploy the workload to multiple locations - Reliability Pillar

https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_fault_isolation_multiaz_region_system.html

28 29 **Well-Architected Reliability:**

https://pages.awscloud.com/rs/112-TZM-766/images/01_Well-Architected%20Reliability%20-%20Best%20practices%20you%20can%20use%20today-WARSAW%20v1.1%20%282%29.pdf

31 34 **Using latency-based routing with Amazon CloudFront for a multi-Region active-active architecture | Networking & Content Delivery**

<https://aws.amazon.com/blogs/networking-and-content-delivery/latency-based-routing-leveraging-amazon-cloudfront-for-a-multi-region-active-active-architecture/>

45 46 47 48 49 50 51 52 **New – Sustainability Pillar for AWS Well-Architected Framework | AWS News Blog**

<https://aws.amazon.com/blogs/aws/sustainability-pillar-well-architected-framework/>